

Lumina Cosmetics & Beauty Store

Live Links

- Live Storefront: <https://lumina-indol-tau.vercel.app>
- Live Backend API: <https://lumina-ecommerce-gmqg.onrender.com>
- GitHub Repository: https://github.com/Nikshan0702/LUMINA_Ecommerce.git
- Business WhatsApp Number: +94771129911

Application Screenshots

Home Page

![Home Page](screenshots/01-home-hero.png)

Products Catalog & Filtering

![Products Catalog](screenshots/02-products-catalog.png)

Shopping Bag & Order Summary

![Shopping Bag](screenshots/03-shopping-bag.png)

Demo Accounts for Testing

You can use these accounts to test the application, or use the one-click demo buttons on the login page:

Admin Account

- Email: admin@lumina.com
- Password: adminpassword123
- Access: Full Admin Dashboard, Product Inventory Management, Order Status & Dispatch updates

Customer Account

- Email: customer@example.com
- Password: customerpassword123
- Access: Browsing, Shopping Bag, Checkout, Past Order History

(You can also register a new account or browse and add items to the cart as a guest before logging in at checkout).

Quick Testing Guide

1. Customer Flow

1. Open the live site at <https://lumina-indol-tau.vercel.app>.
2. Search for items or filter by category (Skincare, Haircare, Makeup, etc.).
3. Open a product page and select quantity (inventory bounds prevent adding more than available stock).
4. Go to the Shopping Bag and proceed to Checkout.
5. Fill in delivery details and choose your payment method:
 - PayHere Online Payment: Opens the PayHere payment modal with verified amount.
6. Completing payment updates the order status to Paid.
 - Order via WhatsApp: Pre-fills a clean, itemized order message with line items, quantities, subtotal, Rs. 500 delivery, and human-readable order code, directly opening a chat with +94771129911.
6. View your placed order under My Orders with real-time status badges.

2. Admin Flow

1. Go to /admin/login (or click Admin Portal).
2. Log in with admin@lumina.com / adminpassword123.
3. View business metrics on the Dashboard (Total Revenue, Orders, Pending Dispatches, Catalog Count).
4. Go to Products to add new items, update stock, change prices, or delete products.
5. Go to Orders to view customer orders, inspect items, update fulfillment (Pending -> Confirmed -> Shipped -> Delivered), or update payment (Pending / Paid).
6. If you change an order's status to Cancelled, the system automatically adds the ordered quantities back to product stock.

Tech Stack

- Frontend: React 18, Vite, Tailwind CSS, React Router v6, Axios, Lucide Icons
- Backend: Node.js, Express.js (REST API)
- Database: MongoDB Atlas with Mongoose
- Authentication: JWT (JSON Web Tokens) with 30-day expiry + bcryptjs password hashing
- Deployment: Vercel (Frontend SPA) + Render (Backend Web Service) + MongoDB Atlas (Database)

Project Structure

```
``text
TaskDartCode/
├── client/                                # Frontend React (Vite)
│   ├── public/                          # Static assets, logo, favicon, _redirects
│   └── src/
│       ├── components/                  # Navbar, Footer, ProductCard, AdminNavbar
│       ├── context/                    # AuthContext, CartContext
│       ├── pages/                      # Storefront pages & admin panel pages
│       ├── services/                   # api.js (Axios instance with JWT interceptor)
│       └── utils/                      # formatters.js (currency, dates, order codes)
│   └── vercel.json                     # SPA routing fallback
└── server/                             # Backend Node.js / Express
    ├── config/                        # db.js (MongoDB connection)
    ├── controllers/                   # auth, product, order, admin controllers
    ├── middleware/                    # JWT auth & admin role guards, error handler
    ├── models/                       # User, Product, Order Mongoose schemas
    ├── routes/                       # Express API routes
    └── utils/                         # payhere hash utility, seed data
``
```

Key Technical Decisions & Security

1. Price Verification on the Backend:

The frontend never tells the server how much an item costs. During checkout, the backend takes the product IDs, fetches their actual prices directly from MongoDB, and calculates the subtotal, delivery fee, and total amount on the server. This prevents any client-side price tampering.

2. Inventory Stock Management:

When an order is created, product quantities are automatically decremented from available stock. If an admin cancels an order, the server loops through the l

ine items and automatically restores the stock.

3. PayHere Hash Generation:

The payment hash is calculated on the server using Node's crypto library following PayHere's formula: MD5(merchant_id + order_id + amount + currency + MD5(merchant_secret)). The merchant secret is stored securely in backend environment variables and is never sent to the browser.

4. Structured WhatsApp Message:

Instead of just sending a link, the checkout builds an itemized, readable message including product names, quantities, unit prices, subtotal, delivery fee, customer details, and a short human-readable order ID (#ORD-XXXXXX).

5. Role-Based Access Control (RBAC):

Admin APIs and pages are protected. Regular customers cannot access administrative endpoints (/api/admin/*, product creation/edits), returning 403 Forbidden if attempted without an admin token.

6. Clean Order Numbers:

Raw 24-character database ObjectIds (like 6ac4b10...) are converted into clean, readable order codes (#ORD-DE1997) across both customer and admin interfaces.

Database Design Summary

- Users: Stores name, email (unique), hashed password, phone, and role (customer or admin).
- Products: Stores name, description, category, brand, price, image URL, stock count, and active visibility toggle.
- Orders: References the user and stores customer info, line items snapshot (product, name, price, quantity, image), subtotal, delivery fee, total, payment method (PayHere / WhatsApp), payment status (Pending / Paid), and fulfillment status (Pending, Confirmed, Shipped, Delivered, Cancelled).

Local Setup

If you wish to run the project locally:

1. Backend

```
```bash
cd server
npm install
```

Create a server/.env file:

```
```env
PORT=5001
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
PAYHERE_MERCHANT_ID=1211149
PAYHERE_SECRET=your_payhere_secret
WHATSAPP_NUMBER=94771129911
NODE_ENV=development
```

Seed initial products and accounts:

```
```bash
npm run seed
```

```
Start backend:
```bash
npm run dev
# Running on http://localhost:5001
```
```

```
2. Frontend
```bash
cd ../client
npm install
npm run dev
# Running on http://localhost:5173
```
```

---

## ## Assumptions & Limitations

- Currency: Set to Sri Lankan Rupees (LKR).
- Delivery Fee: Flat rate of Rs. 500 across Sri Lanka.
- PayHere: Implemented using standard PayHere Sandbox test merchant credentials.
- WhatsApp: The WhatsApp flow opens WhatsApp Web or the WhatsApp app with the message pre-filled; the customer taps send to complete direct communication.